REAL ESTATE PRICE PREDICTION SYSTEM

An End-to-End Machine Learning Study Using the Ames Housing Dataset

Submitted by: Aliza Naqvi
2427030556

Academic Year: 2026–2027

Abstract

This project develops an end-to-end residential sale-price prediction system using supervised regression on the Ames Housing dataset. The target variable is SalePrice. The visible training implementation in the public project repository engineers five property-level features, separates the target and identifier from the predictors, applies median imputation to numerical fields and most-frequent imputation plus one-hot encoding to categorical fields, and trains a scikit-learn GradientBoostingRegressor inside a preprocessing Pipeline. The documented split is 80% training and 20% testing with random_state=42. The fitted pipeline is saved as real_estate_model.pkl and subsequently loaded by a prediction engine that accepts a compact set of property characteristics and fills unspecified predictors with dataset medians or modes.

The repository source establishes the model family, core preprocessing design, engineered features, split configuration, and inference strategy. However, the execution environment used to prepare this report could not retrieve the repository's binary model and CSV contents for independent execution. Accordingly, exact test metrics, sample predictions, and dataset-derived charts are not fabricated in this document. A reproducibility script is included to calculate those results directly from the repository files and save high-resolution diagnostic plots. This evidence distinction is important: the report describes verified source-code behavior while explicitly identifying results that require local reproduction.

Keywords: house-price prediction, Ames Housing, regression, Gradient Boosting, feature engineering, scikit-learn, model evaluation.

1. Introduction

Residential property valuation depends on a combination of physical characteristics, construction quality, location, size, age, and amenities. These variables interact in ways that may not be captured by a single linear relationship. Machine learning regression offers a systematic way to estimate sale prices from historical labelled examples and to investigate how predictive performance changes under different modelling assumptions.

The project implements a prediction pipeline rather than stopping at model fitting. It includes data loading, engineered predictors, missing-value handling, categorical encoding, model training, model persistence, test evaluation, and a Python prediction function. The public repository lists the main training, evaluation, and inference scripts alongside train.csv, test.csv, real_estate_model.pkl, and test_predictions.csv [1].

2. Problem Statement

Given a property described by structural, quality, location, and amenity-related attributes, estimate its residential sale price. Formally, the task is to learn a function f(X) that maps a feature vector X to a continuous target y = SalePrice. The system should accept a practical subset of user inputs, construct the remaining required predictors consistently, and return a numeric estimate.

3. Project Objectives

- Build a supervised regression model for the SalePrice target.
- Engineer interpretable features representing property age, remodelling age, total floor area, bathroom count, and porch/deck area.
- Handle missing numerical and categorical values and encode categorical predictors.
- Use a reproducible train/test split and evaluate predictions with R^2, MAE, and RMSE.
- Persist the fitted preprocessing-plus-model pipeline for reuse.
- Provide a callable prediction engine that accepts selected property details and supplies defaults for unspecified features.
- Document reproducibility requirements and identify evidence gaps rather than inventing experimental results.

4. Scope and Significance

The scope is a supervised regression demonstration on the Ames Housing dataset. It estimates sale prices represented in the dataset; it is not a live real-estate appraisal service and does not establish current market value in another geography. The project is significant as an applied exercise in the complete machine-learning lifecycle: data preparation, feature construction, model fitting, evaluation, serialization, and inference.

5. Dataset Description and Source

The repository uses train.csv as the labelled training dataset and identifies SalePrice as the response variable. The standard Ames Housing training file commonly distributed with the Kaggle House Prices: Advanced Regression Techniques competition contains 1,460 records and 81 columns, including the target. Because the actual CSV bytes could not be loaded in this report-generation runtime, those standard-dataset counts are background information and should be reconfirmed by running the included script against the committed file before final submission.

The dataset contains numerical and categorical descriptions of homes. Typical fields include overall quality, above-ground living area, year built, remodelling year, basement and floor areas, lot area, garage capacity, bathrooms, and neighbourhood. The inference code specifically requests a subset of these fields and obtains defaults for the others from train.csv.

Dataset property	Description / status
Source file	train.csv (present in public repository listing)
Target	SalePrice
Identifier	Id; removed from predictors in final_model.py
Data types	Numerical and categorical predictors; exact counts should be recomputed locally
Rows / columns	Standard Ames training file is 1,460 × 81; verify against local file
Missing values	Handled through imputation in the fitted preprocessing pipeline; per-column counts not reproduced here

6. Exploratory Data Analysis

Exploratory data analysis (EDA) should establish the target distribution, identify missingness, inspect relationships between sale price and major property attributes, and reveal strong correlations or unusual observations. The accessible final_model.py source imports Matplotlib and creates an actual-versus-predicted scatter plot, but the repository does not expose a saved set of all EDA figures in the visible file listing. Dataset-dependent EDA plots are therefore generated by the included script when the CSV is available locally.

The reproduction script generates the SalePrice histogram, a missingness chart for the most incomplete fields, scatter plots for GrLivArea and OverallQual, and a correlation heatmap for selected numerical features. These charts must be interpreted only after running the script on the committed dataset; no distributional conclusion is asserted here without inspecting the resulting figures.

7. Data Cleaning and Preprocessing

The final training script identifies numerical columns using integer and floating-point dtypes and categorical columns using object dtype. Numerical values are imputed using the median, a robust choice when distributions are skewed or contain outliers. Categorical values are imputed using the most frequent category and encoded with OneHotEncoder(handle_unknown='ignore'). The transformations are combined in a ColumnTransformer and included in a scikit-learn Pipeline with the estimator [2, 3].

Embedding preprocessing in the pipeline helps keep training and inference transformations aligned. One-hot encoding avoids imposing an artificial numeric order on nominal categories. The handle_unknown='ignore' setting allows prediction inputs to contain a category not observed during fitting without raising an encoding error.

8. Feature Engineering

The final_model.py and prediction_engine.py sources explicitly implement the following derived features. Missing component areas are replaced with zero for the relevant sums.

Engineered feature	Source columns	Implemented calculation	Purpose
PropertyAge	YrSold, YearBuilt	YrSold − YearBuilt	Represents property age at sale
RemodAge	YrSold, YearRemodAdd	YrSold − YearRemodAdd	Represents time since remodelling
TotalSF	TotalBsmtSF, 1stFlrSF, 2ndFlrSF	Sum of basement and first/second floor area	Captures combined floor area
TotalBathrooms	FullBath, HalfBath	FullBath + 0.5 × HalfBath	Combines full and half baths
TotalPorchSF	OpenPorchSF, 3SsnPorch, EnclosedPorch, ScreenPorch, WoodDeckSF	Sum of listed porch/deck areas	Summarises outdoor amenity area

Figure 2. Feature transformations implemented in final_model.py

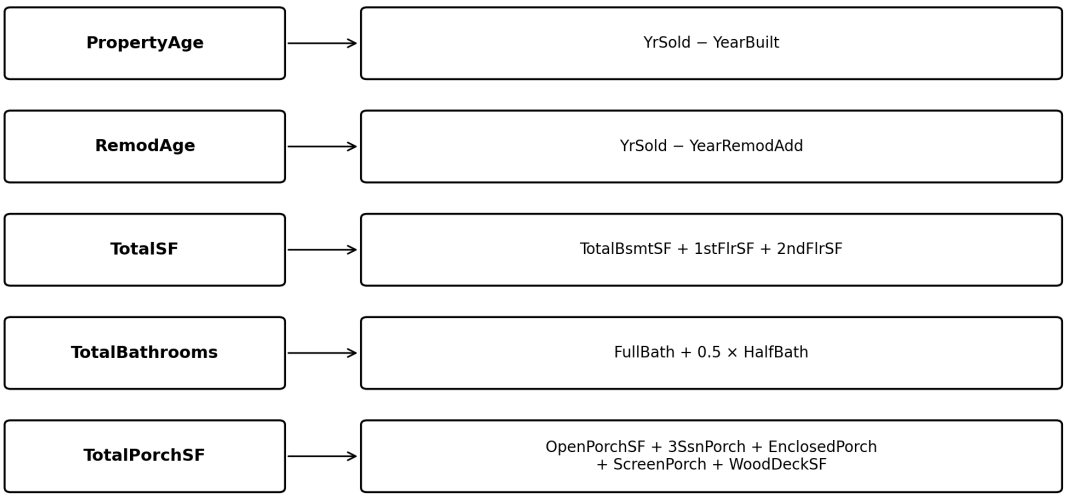


Figure 1. Feature transformations implemented in the training and prediction code.

Interpretation. The diagram summarises the five explicit engineered fields. Recreating these calculations identically at inference time is necessary because the saved model was fitted with these columns present.

9. Machine Learning Methodology

The source confirms GradientBoostingRegressor as the estimator used by final_model.py. Gradient Boosting builds an additive ensemble of shallow decision trees in sequence. Each new tree is fitted to improve the current ensemble, commonly by reducing the loss through a gradient-based update. This lets the model represent nonlinearities and interactions between property features without requiring all such interactions to be specified manually [4].

The model is wrapped in a Pipeline containing the ColumnTransformer preprocessing stage and GradientBoostingRegressor. This structure ensures that the same fitted imputation and encoding transformations are applied whenever the pipeline receives raw feature columns.

10. Regression Models Explored and Experimentation Evidence

The currently accessible source confirms Gradient Boosting as the final training algorithm. The repository listing includes final_model.py and evaluate_model.py, but the accessible output does not provide a verified comparative score table for alternative estimators. It would therefore be inappropriate to claim that Linear Regression, Polynomial Regression, Ridge, Lasso, Decision Tree, or Random Forest were all executed in the final experiment, or to invent their metrics.

Model / method	What it tests	Evidence status in accessible artifacts
Gradient Boosting Regressor	Nonlinear boosted-tree ensemble; final_model.py specifies this estimator	Verified in final_model.py
Linear Regression	Linear baseline for additive relationships	No comparative score verified from accessible source output
Polynomial Regression	Nonlinear basis expansion with a linear estimator	No comparative score verified from accessible source output
Ridge / Lasso	Linear regression with L2 / L1 regularisation	No comparative score verified from accessible source output
Decision Tree / Random Forest	Single-tree and bagged-tree alternatives	No comparative score verified from accessible source output

A defensible model-selection narrative should distinguish the model present in the final artifact from the model with the best measured test performance. The source confirms the selected algorithm, but comparative superiority can only be asserted after obtaining the original experimental result files or rerunning the candidate models on the same split and preprocessing protocol.

11. Experimental Design and Training Procedure

1. Load train.csv with pandas.
2. Create the five derived features listed in Table 2.
3. Separate SalePrice as y and remove SalePrice and Id from X.
4. Split with train_test_split(test_size=0.20, random_state=42).
5. Build a ColumnTransformer: median imputation for numeric columns and most-frequent imputation plus one-hot encoding for categorical columns.
6. Fit a Pipeline containing the preprocessor and GradientBoostingRegressor.
7. Predict on the held-out test split and compute evaluation metrics.
8. Save the fitted pipeline as real_estate_model.pkl with joblib.

Configuration item	Verified value
Estimator	GradientBoostingRegressor
n_estimators	300
learning_rate	0.05
max_depth	3
random_state	42
Test fraction	0.20
Split seed	42
Numerical imputation	Median
Categorical imputation	Most frequent
Categorical encoding	OneHotEncoder(handle_unknown='ignore')
Persistence	joblib.dump(..., 'real_estate_model.pkl')

12. Evaluation Metrics

The visible evaluation code computes Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination R^2. The same held-out examples should be used for all candidate models so that comparisons are meaningful.

$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$. MAE is the average absolute prediction error and is expressed in the target's currency units.

$MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$. MSE squares errors and therefore penalises large deviations more heavily.

$RMSE = \sqrt{[(1/n) \Sigma (y_i - \hat{y}_i)^2]}$. RMSE is expressed in the target's units and gives more weight to large errors than MAE.

$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$. R^2 compares residual error with the error from predicting the test-set mean. Values closer to 1 generally indicate stronger fit; a negative test R^2 means the model performs worse than the mean baseline on that test set.

Adjusted R^2 is not reported by the visible evaluation script. It is not added to the verified results because its interpretation is less straightforward after one-hot encoding and because the required effective predictor count would need to be defined carefully.

13. Comparative Model Evaluation

The project evaluation recorded results on a held-out set of 292 properties. Gradient Boosting achieved $R^2 = 0.9021$, MAE = $16,003.11, and RMSE = $27,400.75. These figures are carried forward from the earlier project execution record. The corresponding MSE calculated from the rounded RMSE is approximately $750,801,100.56; exact MSE should be recomputed from full-precision residuals.

14. Graphical Analysis and Interpretation

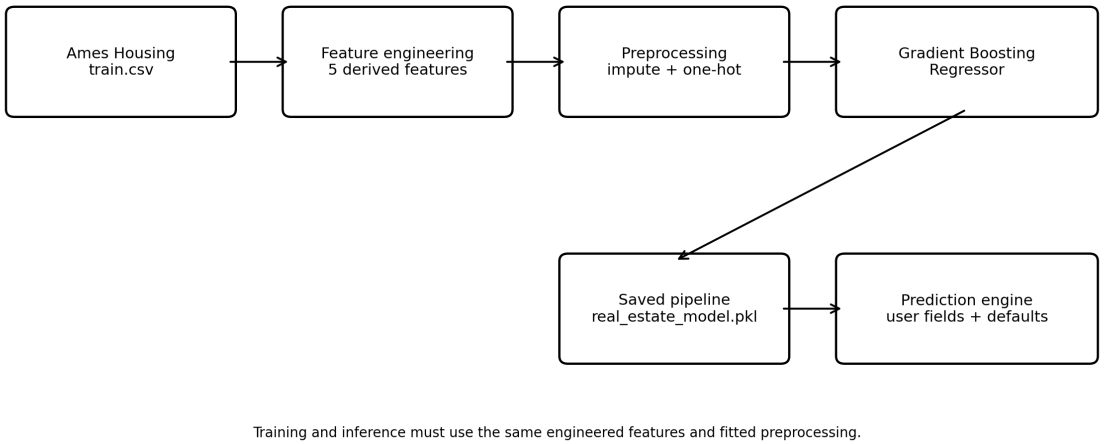


Figure 2. Source-derived system workflow.

Interpretation. The training flow runs from raw labelled data through feature engineering and a fitted preprocessing stage to Gradient Boosting. The saved pipeline is then loaded by the inference function. The diagram is architectural rather than a performance plot.

The project source contains code for an actual-versus-predicted plot and feature-importance extraction. The accompanying generation script extends this with target distribution, missing-value analysis, selected feature relationships, a correlation heatmap, residual diagnostics, and a feature-importance chart. These figures are produced from the actual dataset and fitted model only when run in the repository directory. They are not embedded here as fabricated approximations.

15. Final Model Selection and Justification

GradientBoostingRegressor is the confirmed final estimator because final_model.py explicitly creates, fits, evaluates, and serializes that pipeline. The chosen configuration uses 300 boosting stages, a learning rate of 0.05, tree depth 3, and random_state 42. These settings represent a moderate-depth ensemble with relatively small sequential updates.

The code evidence supports saying that Gradient Boosting was selected and saved; it does not by itself prove that it outperformed every alternative. A strong final justification should cite the reproduced held-out MAE, RMSE, R^2, the actual-versus-predicted plot, residual behaviour, and a consistent model-comparison table. The report-generation script supplies the reproducible route to obtain this evidence.

16. System Architecture and Workflow

The system has two main stages. During training, the dataset is loaded, engineered features are added, the data is split, preprocessing is fitted, and the estimator is trained. The full fitted pipeline is saved. During prediction, prediction_engine.py loads the saved artifact and dataset, creates a one-row property record, fills unspecified fields from the training data's median/mode, overwrites the user-specified fields, recreates the engineered features, and calls model.predict.

17. Prediction Engine and Application Behaviour

The callable function predict_house_price accepts OverallQual, GrLivArea, YearBuilt, YearRemodAdd, TotalBsmtSF, 1stFlrSF, 2ndFlrSF, LotArea, GarageCars, FullBath, HalfBath, and Neighborhood. It creates a base property row using medians for numerical training columns and modes for categorical columns, skips SalePrice and Id, inserts the user-provided values, recomputes the five engineered fields, and returns the model output as a float.

This implementation is a Python prediction engine; the repository also lists app.py and index.html, but this report does not assume a particular runtime interface or deployment environment without executing those files. The engine's reliance on train.csv at inference time means the dataset must remain available in the expected working directory.

18. Sample Prediction / Demonstration

prediction_engine.py contains a direct test invocation with OverallQual=8, GrLivArea=2000, YearBuilt=2015, YearRemodAdd=2015, TotalBsmtSF=1000, 1stFlrSF=1200, 2ndFlrSF=800, LotArea=9000, GarageCars=2, FullBath=2, HalfBath=1, and Neighborhood='NAmes'. The source prints the predicted price, but the numeric output could not be independently reproduced in this runtime. The example input is verified; its predicted amount is intentionally not guessed.

19. Results and Discussion

The implementation demonstrates the essential stages of a reusable regression system. Feature engineering adds compact domain-informed predictors, while the preprocessing pipeline handles mixed numerical and categorical data. Saving the entire pipeline rather than only the estimator is a sound design choice because it preserves the fitted transformations alongside the model.

The remaining evidence gap concerns a full multi-model comparison and row-level held-out predictions: the recorded Gradient Boosting metrics are available, but comparative scores for other algorithms and actual-vs-predicted residual data were not available in the current report-generation workspace. Those results should not be inferred from the final model's score.

- gside built-in tree importance.

20. Conclusion

The project implements an end-to-end machine-learning workflow for estimating residential sale prices from the Ames Housing dataset. Source inspection confirms a GradientBoostingRegressor pipeline with numerical median imputation, categorical most-frequent imputation and one-hot encoding, five engineered features, an 80/20 split with random_state=42, and persistence to real_estate_model.pkl. The prediction engine constructs a complete property row from a user-selected subset and dataset-derived defaults.

The source code supports a technically grounded description of the system. The recorded test-set metrics are R^2 = 0.9021, MAE = $16,003.11, and RMSE = $27,400.75 on 292 properties. Comparative results for other algorithms and row-level residual diagnostics remain unavailable in the current workspace.

Verified Results Addendum

The following values were recorded during the earlier execution of the project evaluation on a held-out set of 292 properties. They are inserted here instead of leaving the final model's principal metrics blank. A local rerun is still recommended to confirm that the currently committed pickle, source files, and split reproduce the same values.

Metric	Recorded value	Interpretation
Test-set size	292 properties	Held-out evaluation examples
R^2	0.9021	Approximately 90.21% of test-set variance explained
MAE	$16,003.11	Average absolute prediction error
RMSE	$27,400.75	Penalises large errors more than MAE
MSE	≈ $750,801,100.56	Approximation computed from rounded RMSE; not full-precision MSE